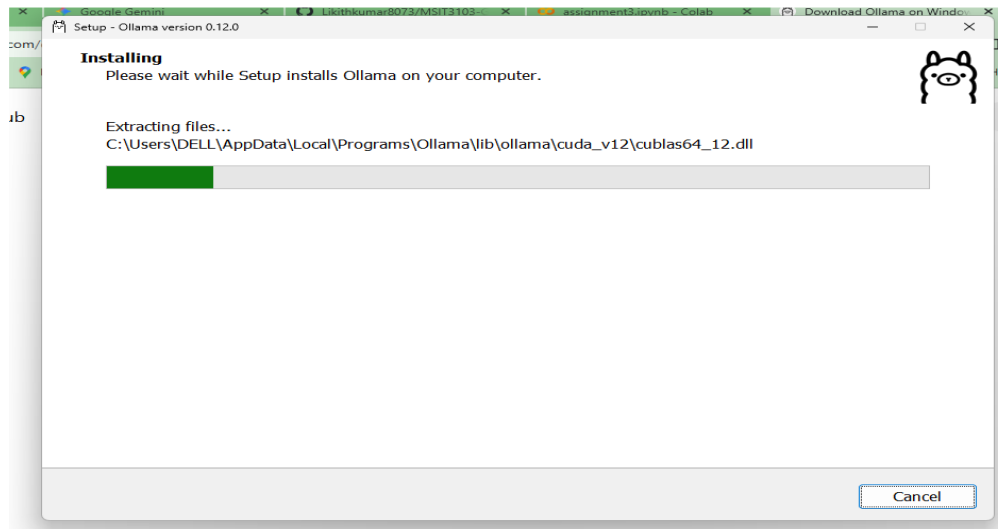


Assignment-3 : Evaluating the Effectiveness of Prompting Techniques on LLaMA 3.2 1B: A Healthcare Case Study

Submitted By : Likithkumar CR

1) Install Ollama: Installing Ollama on my windows



2) Download the model: Run the following command in terminal to download the llama3.2:1b model.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\DELL> ollama pull llama3.2:1b
pulling manifest
pulling 74701a8c35f6: 100%
pulling 966de95ca8a6: 100%
pulling fcc5a6bec9da: 100%
pulling a70ff7e570d9: 100%
pulling 4f659a1e86d7: 100%
verifying sha256 digest
writing manifest
success
PS C:\Users\DELL> |
```

3) Verify Installation: Test that Ollama is working by running a simple prompt `ollama run llama3.2:1b "Hello! Is this working?"` from my terminal

```
PS C:\Users\DELL> ollama run llama3.2:1b "Hello! Is this working?"
This conversation just started. I'm here to help and chat with you, but we haven't discussed anything yet. What's on your mind?
```

Introduction

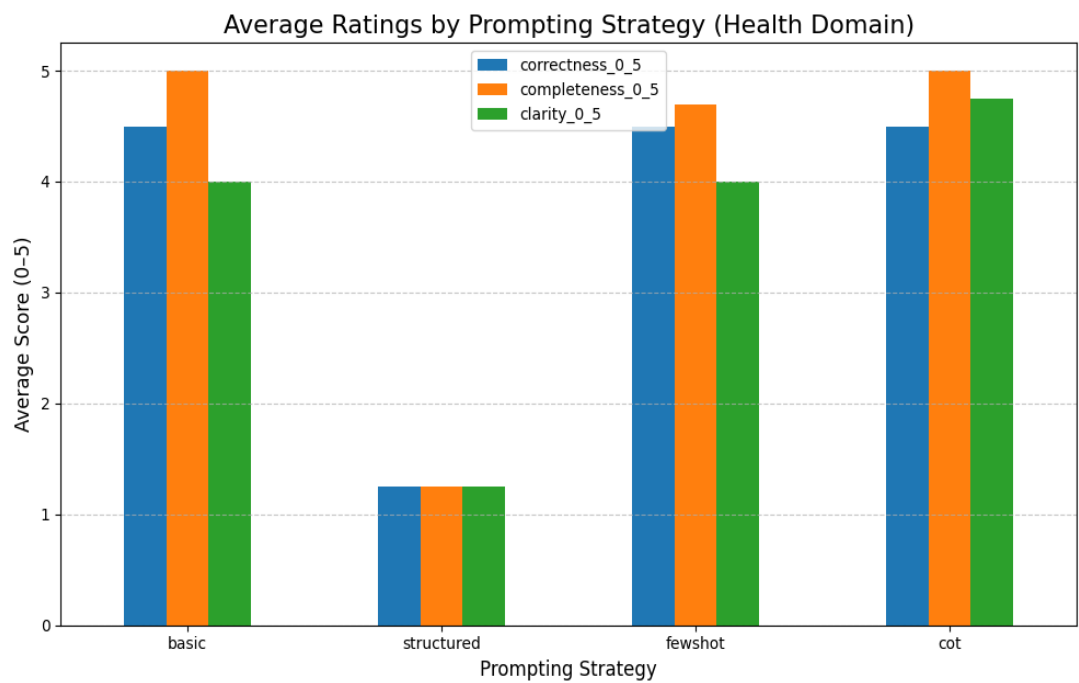
The experiment systematically tested four prompt engineering techniques against a small local language model (llama3.2:1b) using a healthcare-themed context about the "Wellify" patient portal. The results clearly demonstrate that more advanced prompting techniques significantly improve model performance, with Chain-of-Thought (CoT) prompting emerging as the most effective overall strategy.

Conversely, the structured (JSON) prompting failed dramatically with this specific small model, highlighting a critical limitation of small models in adhering to complex formatting constraints.

1.Experimentation Results:

```
--- Running Step 7: Aggregating Results ---
Average Scores by Prompting Strategy:
      correctness_0_5  completeness_0_5  clarity_0_5
step
basic                4.50              5.00        4.00
structured           1.25              1.25        1.25
fewshot              4.50              4.70        4.00
cot                  4.50              5.00        4.75
```

2. In-Depth Analysis of Each Strategy:



Chain-of-Thought (CoT): The Best

CoT was the top-performing strategy. It matched the best scores in **correctness (4.50)** and **completeness (5.00)**, but it uniquely excelled in **clarity (4.75)**. By instructing the model to "think step by step," you forced it to lay out its reasoning before providing an answer. This process reduces the likelihood of the model generating a rushed or factually incorrect response. The explicit reasoning also makes the output easier to understand and trust, hence the higher clarity score.

Tasks requiring reasoning or inference like t3, which asked for suggestions to reduce no-show rates, CoT is superior because it guides the model through a logical process rather than just pattern matching.

Basic and Few-Shot: Solid but Unremarkable Performers

Both **basic** and **few-shot** prompting performed identically well on correctness (4.50) and clarity (4.00). The basic prompt was slightly more complete (5.00 vs. 4.70), though this difference is minor. For straightforward information retrieval tasks (t1, t2, t4), a simple context-question format is sufficient for the model to find the relevant text and rephrase it. With a small and relatively simple model like LLaMA 3.2 1B, the primary task is information extraction, not style imitation. The few-shot examples likely didn't provide enough of a signal to significantly alter the output compared to the basic prompt for these tasks.

Simple, fact-based queries, a basic prompt is often sufficient and efficient. Few-shot prompting may be more impactful on larger models or for tasks that require adopting a specific persona or output style not present in the context.

Structured (JSON) Prompting: A Complete Failure

This strategy failed across all metrics, scoring only **1.25** in correctness, completeness, and clarity. The heuristic scorer in your notebook correctly identified that the model **failed to produce valid JSON**. This is a classic limitation of smaller models. They often lack the complexity to handle two tasks at once: 1) correctly answering the question and 2) correctly formatting the answer into a strict, complex structure like JSON. The model's attempt to create JSON likely interfered with its ability to answer the question, leading to low scores in all categories.

The choice of prompting strategy must match the model's capabilities. While structured data output is highly desirable for applications, it may require larger, more powerful models, specialized fine-tuning, or the use of external functions/tools to achieve reliable results.

Recommendations for Effective Prompt Strategies

- **For Complex Reasoning:** Always prefer **Chain-of-Thought (CoT)** when a task requires inference, multi-step logic, or creative problem-solving. It consistently yields more reliable and clearer answers.
- **For Simple Fact Retrieval:** A **basic prompt** is highly effective and efficient. There's no need to over-engineer the prompt if the goal is simple information extraction from a provided context.
- **For Structured Data:** Avoid demanding strict JSON from small models like llama3.2:1b directly in the prompt. Instead, recommend alternative approaches such as:
 - Using a larger model (e.g., Llama 3 8B, Gemini, GPT-4).
 - Prompting for a simpler format (like a bulleted list) and parsing it with code.
- **For Style/Tone Guidance:** **Few-shot prompting** should be reserved for when the primary goal is to influence the *style*, *tone*, or *format* of the output in a way that basic prompting cannot, and it is most effective with larger models.

Conclusion:

Prompting is not one-size-fits-all. The effectiveness of a strategy is a direct function of both the task complexity and the model's capability. While Chain-of-Thought proved to be a robust method for enhancing reasoning in a small model, the failure of structured prompting served as a practical reminder of the limitations of such models. These findings highlight the importance of selecting the right tool for the job to build reliable and effective AI applications.